CHAPTER -1

INTRODUCTION

1.1 Introduction

Software development has evolved tremendously over the decades, incorporating cutting-edge frameworks, methodologies, and best practices. However, despite these advancements, the process remains inherently vulnerable to various forms of human error. These errors, commonly referred to as bugs, can manifest in multiple ways — from simple syntax mistakes like missing semicolons or incorrect indentation, to more intricate logical flaws that subtly undermine the intended behavior of a program. Such bugs, regardless of their nature, have the potential to severely compromise the reliability, performance, and security of software systems, often leading to costly failures or vulnerabilities. Traditionally, the task of identifying and fixing these issues, known as debugging, has been a manual, time-consuming, and often error-prone process. It requires a combination of deep technical expertise, acute problem- solving skills, and considerable patience. Developers must painstakingly comb through extensive lines of code, run numerous test cases, and perform step-by-step analyses to trace the root cause of errors—a task that becomes even more daunting as systems grow in size and complexity.

In recent years, the rapid advancement of Artificial Intelligence (AI)—especially in fields like machine learning (ML) and natural language processing (NLP)—has introduced a paradigm shift. AI models are now capable of understanding, analyzing, and even generating code with impressive accuracy. This technological leap has opened new possibilities for automating and optimizing the debugging process, dramatically reducing the time and effort required while enhancing precision and efficiency. One such innovation is BugBuster, an intelligent system specifically engineered to assist developers at multiple stages of the software lifecycle. BugBuster is designed not only to detect and diagnose bugs with high accuracy but also to suggest corrections and, in some cases, even generate code autonomously. By integrating AI-driven tools like BugBuster into development workflows, teams can streamline their processes, accelerate delivery timelines, and improve overall software quality. This convergence of AI and software engineering marks a significant step toward more resilient, efficient, and intelligent development ecosystems.

1.2 Overview of the Project

BugBuster is an AI-powered automated debugging system developed to enhance the efficiency and reliability of modern software development. The primary objective of this project is to reduce the manual effort involved in detecting, diagnosing, and correcting software bugs by leveraging advanced techniques from machine learning and natural language processing. BugBuster analyzes source code to identify various types of errors, including syntax errors, logical flaws, runtime issues, and common coding anti-patterns that often go unnoticed during manual reviews.

The system operates by examining code structure, program flow, and error patterns learned from large datasets of real-world software projects. Once a bug is detected, BugBuster provides detailed diagnostic information explaining the cause of the error and its impact on program execution. In addition, the system suggests appropriate fixes and, where applicable, automatically generates corrected code snippets. This assists developers in understanding mistakes while significantly accelerating the debugging process. BugBuster is designed to support multiple programming languages and integrates seamlessly with existing development environments. It can be used during various stages of the software development lifecycle, including coding, testing, and maintenance. By automating repetitive debugging tasks, the system allows developers to focus more on design and innovation rather than error resolution.

Overall, the BugBuster project demonstrates how artificial intelligence can be effectively applied to software engineering to create smarter development tools. The proposed system aims to improve code quality, minimize development time, reduce human error, and contribute to the creation of more robust, secure, and maintainable software systems.

1.3 Problem Statement

Bug Buster is an intelligent, cross-language system designed to assist developers by detecting and fixing code errors in real time. The system aims to enhance coding efficiency, reduce errors and boost overall developer productivity.

1.4 Scope of the Project

The scope of the BugBuster project is broad and multifaceted, addressing various aspects of software development and debugging:

- Multi-Stage Support
- Bug Detection and Correction
- Multi-Language Capability
- Educational Utility

1.5 Objectives of the Project

- **Automated Error Detection**

➢ **Objective:** Detect syntax, logical, and runtime errors in real-time during code development.

➢ **Algorithms/Models Used:**

- **Graph Neural Networks (GNNs):** Analyse Abstract Syntax Tree (ASTs) and control/data flow to detect structural and logical errors.
- **Transformers (e.g., DeepSeek-Coder:1.3b):** Understand code semantics and flag contextual errors.

- **Automated Code Correction**

➢ **Objective:** Suggest or apply fixes to erroneous code while preserving the logic and structure.

➢ **Algorithms/Models Used:**

- **DeepSeek-Coder:1.3b:** Generate corrected code using sequence-to-sequence prediction.
- **Decision Trees:** Identify appropriate corrections based on error types.

- **Multi-Language Support**

➢ **Objective:** Compile the programs written in multiple programming languages.

➢ **Algorithms / Models Used:**

- **Language-Specific Parsers & AST Generators:** Construct Abstract Syntax Trees for different languages (such as C, C++, Java, Python) to support syntax analysis and structural understanding.
- **Unified Intermediate Representation (IR):** Converts language-specific code into a common representation to apply shared fault localization and repair logic across languages.
- **Rule-based Language Profiles:** Apply language-specific grammar rules, compiler constraints, and coding standards to ensure accurate analysis and patch generation.
- **Machine Learning Classifiers (SVM / Random Forests):** Classify buggy patterns across different programming languages using learned features from multilingual code datasets.

CHAPTER-2

LITERATURE SURVEY

1. DEAR: A Novel Deep Learning-based Approach for Automated Program Repair

Publication: 2024

Authors: Yi Li, Shaohua Wang, Tien N. Nguyen

a. Abstract

DEAR (Deep learning-based Approach for Error Repair) is an advanced automated program repair (APR) technique developed to overcome the limitations of traditional APR models, which often struggle with complex software bugs. Unlike conventional models that typically focus on single-line or isolated fixes, DEAR is capable of handling general defects that require multiple dependent changes spread across different statements and code hunks. It leverages deep learning to understand the broader context of code and generate coherent, multi-location patches. This makes DEAR especially effective in repairing real-world bugs where logical relationships and dependencies span across non-contiguous code regions, enhancing the reliability of automated bug fixing.

b. Key Contributions

- **Fault Localization Technique:** Combines traditional spectrum-based fault localization (SBFL) with deep learning and data-flow analysis to identify buggy statements and hunks that need simultaneous fixing.
- **Two-Tier LSTM Model:** Utilizes a tree-based Long Short-Term Mem ory (LSTM) model enhanced with attention layers and cycle training to learn code transformations effectively.
- **Extensive Evaluation:** DEAR was tested on three datasets (Defects4J, BigFix, CPatMiner) and showed significant improvements over existing models, fixing 31% to 683% more bugs depending on the dataset.

c. Algorithms and Models Used

- **Spectrum-Based Fault Localization (SBFL):** Identifies suspicious buggy statements based on execution data.
- **Tree-based LSTM Model:** A two-tier model that processes Abstract Syntax Trees (ASTs) to learn transformations for fixing code.

d. Limitations/Drawbacks

- **Dependency on Training Data:** The performance of DEAR heavily relies on the quality and quantity of training data.
- **Complexity of Model:** The two-tier LSTM model may introduce complexity in training and inference, requiring significant computational resources.
- **Limited to Specific Bug Types:** While DEAR improves upon existing models, it may still struggle with certain complex bugs that require more nuanced understanding beyond the training data.

2. Program Repair by Fuzzing over Patch and Input Space

Publication: 2024

Authors: Yuntong Zhang, Ridwan Shariffdeen, Gregory J. Duck, Jiaqi Tan, Abhik Roychoudhury

a. Abstract

This paper presents a novel automated program repair (APR) technique called **patch-level fuzzing**, which innovatively combines fuzz testing with patch-level exploration to enhance both the efficiency and effectiveness of the repair process. Traditional APR methods often require recompilation after applying each patch, which can be time-consuming and computationally expensive. Patch-level fuzzing addresses this challenge by eliminating the need for recompilation during patch validation. Instead, it dynamically evaluates patch correctness using fuzz-generated inputs, allowing rapid feedback on the validity of candidate patches.

b. Key Contributions

- **Patch-Level Fuzzing:** Adapts traditional fuzzers to explore the patch space instead of
- just the input space, allowing for a more efficient search for plausible patches.
- **Compilation-Free Patch Validation:** Implements a methodology that avoids recompilation by interpreting only the patched statements, signifi cantly improving throughput.
- **Co-Exploration of Spaces:** Combines input-level and patch-level fuzzing to enhance the quality of patches and reduce overfitting.

c. Algorithms and Models Used

- **AFL (American Fuzzy Lop):** A well-known fuzzer adapted for patch level exploration.
- **RNN Model:** Used for classifying statements as buggy or not during the multi-statement expansion process.

d. Limitations/Drawbacks

- **Limited Scope of Fuzzing:** While effective, fuzzing may not cover all edge cases, potentially missing certain bugs.
- **Overfitting Risk:** The approach may still generate patches that work for the test cases but fail in other scenarios, leading to overfitting.
- **Dependency on Input Quality:** The effectiveness of fuzzing is Contin gent on the quality of the input test cases provided.

3. ProveNFix: Temporal Property-Guided Program Re pair

Publication: 2024

Authors: Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, Abhik Roychoud hurry

a. Abstract

ProveNFix is an automated static analysis tool specifically designed to detect and repair temporal property violations in software programs. Temporal properties describe how a system should behave over time, and violations of these properties can lead to serious functional or safety issues. ProveNFix introduces a proof-based repair strategy, where formal proofs derived from temporal logic are used to guide the repair process. Instead of relying solely on pattern matching or machine learning, it constructs logical proofs to understand the root cause of violations and generate correct patches. This approach ensures that the repairs are not only automated but also semantically sound and verifiable.

b. Key Contributions

- **Future-Condition Specification:** Allows users to express expected behaviours after function calls, enhancing the expressiveness of temporal properties.
- **Compositional Static Analysis:** Automatically infers specifications for procedures based on user-supplied specifications for primitive APIs, facilitating modular analysis.
- **Repair Strategy:** Utilizes a term rewriting system to check actual behaviours against inferred specifications, enabling the automatic generation of patches for various bug types.

c. Algorithms and Models Used

- **Compositional Static Analyzer:** Analyses program segments independently to infer specifications.
- **Term Rewriting System:** Used for checking the inclusion of actual behaviors against inferred specifications.

d. Limitations / Drawbacks

- **Complexity of Temporal Properties:** The introduction of future conditions may complicate the specification process for users.
- **Inference Limitations:** The accuracy of inferred specifications depends on the quality of the initial user-supplied specifications.
- **Scalability Issues:** While designed for modular analysis, the approach may face challenges when applied to very large codebases

4. Rethinking AI Code Generation: A One-Shot Correction Approach Based on User Feedback

Publication: 2024

Authors: Kim Tuyen Le, Artur Andrzejak

a. Abstract

This paper introduces a methodology called **"One-shot Correction"**, designed to enhance the user experience in AI-based code generation tools. The core idea is to allow users to provide immediate feedback on incorrect or suboptimal code outputs and have the system generate improved responses in a single interaction—without the need for retraining or fine-tuning the underlying model. One-shot Correction works by dynamically incorporating user-provided hints or corrections into the generation context, enabling the model to adapt its outputs on the fly. This approach significantly improves interactivity, reduces turnaround time for corrections, and makes AI-assisted programming more intuitive and user-centric.

b. Key Contributions

- **Decomposition Techniques:** Breaks down code translation tasks into sub-problems, allowing for more manageable and interpretable code generation.
- **Prototype Development:** A preliminary GUI application was developed to demonstrate the utility of the One-shot Correction methodology.

c. Algorithms and Models Used

- **K-Nearest Neighbors (KNN):** Employed to retrieve similar code snip pets based on user queries.
- **Rule-Based Approach:** Used for chunking and mapping NL queries to code snippets.

d. Limitations/Drawbacks

- **Dependence on User Feedback:** The effectiveness of the approach relies heavily on the quality and quantity of user feedback.
- **Limited Contextual Understanding:** The model may struggle with complex queries that require deeper contextual understanding beyond simple modifications.
- **No Model Retraining:** While avoiding retraining is an advantage, it may limit the model's ability to adapt to new coding patterns or user preferences over time.

5. Automated Program Repair via Fuzzing

Publication: 2024

Authors: Yuntong Zhang, Gregory J. Duck, Jiaqi Tan, Abhik Roychoudhury.

a. Abstract

This paper proposes an innovative approach to **Automated Program Repair (APR)** known as **"patch-level fuzzing,"** which effectively integrates **fuzz testing** with **patch-level exploration**. Unlike traditional repair methods that require recompiling the program after each patch application, this method **bypasses recompilation**, significantly reducing validation time and computational overhead. This technique enhances the **efficiency**, **scalability**, and **fault coverage** of program repair by focusing on runtime behavior rather than static recompilation, making it well-suited for large-scale, real-world software systems.

b. Key Contributions

- **Patch-Level Fuzzing:** Adapts traditional fuzzers to explore the patch space instead of just the input space, allowing for a more efficient search for plausible patches.
- **Compilation-Free Patch Validation:** Implements a methodology that avoids recompilation by interpreting only the patched statements, significantly improving throughput.
- **Co-Exploration of Spaces:** Combines input-level and patch-level fuzzing to enhance the quality of patches and reduce overfitting.

c. Algorithms and Models Used

- **AFL (American Fuzzy Lop):** A well-known fuzzer adapted for patch level exploration.
- **RNN Model:** Used for classifying statements as buggy or not during the multi-statement expansion process.

d. Limitations/Drawbacks

- **Limited Scope of Fuzzing:** While effective, fuzzing may not cover all edge cases, potentially missing certain bugs.
- **Overfitting Risk:** The approach may still generate patches that work for the test cases but fail in other scenarios, leading to overfitting.
- **Dependency on Input Quality:** The effectiveness of fuzzing is contingent on the quality of the input test cases provided.

6. Program Repair via Temporal Property-Guided Analysis

Publication: 2024

Authors: Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin

a. Abstract

This paper presents **ProveNFix**, a static analysis tool designed to automatically detect and repair **temporal property violations** in software programs. ProveNFix introduces a **proof-based repair strategy**, where logical reasoning and formal verification techniques are used to identify the root cause of violations and guide the generation of correct fixes. By leveraging **temporal logic proofs**, the tool ensures that the repairs are not only automated but also **sound**, **reliable**, and **compliant** with specified program properties, enhancing overall software correctness and safety.

b. Key Contributions

- **Future-Condition Specification:** Introduces a new way to express expected behaviours after function calls, enhancing the expressiveness of temporal properties.
- **Compositional Analysis:** Automatically infers specifications for procedures based on user-supplied specifications for primitive APIs.
- **Repair Strategy:** Uses a term rewriting system to check actual behaviours against inferred specifications, enabling automatic patch generation.

c. Algorithms and Models Used

- **Compositional Static Analyzer:** Analyses program segments independently to infer specifications.
- **Term Rewriting System:** Used for checking the inclusion of actual behaviours against inferred specifications.

d. Limitations/Drawbacks

- **Complexity of Temporal Properties:** The introduction of future conditions may complicate the specification process for users.
- **Inference Limitations:** The accuracy of inferred specifications depends on the quality of the initial user-supplied specifications.
- **Scalability Issues:** While designed for modular analysis, the approach may face challenges when applied to very large codebases.

7. FuzzRepair: A Fuzzing-Based Approach to Program Repair

Publication: 2024

Authors: Yuntong Zhang, Gregory J. Duck, Jiaqi Tan, Abhik Roychoudhury

a. Abstract

FuzzRepair is an advanced automated program repair technique that integrates **fuzz testing** with **patch-level exploration** to enhance the speed and effectiveness of the repair process. Traditional APR methods often require recompiling the entire program for each candidate patch, which can be time-consuming and resource- intensive. FuzzRepair overcomes this limitation by introducing a **compilation-free validation mechanism**, allowing patches to be tested directly at the binary or intermediate level using fuzz-generated inputs. This approach accelerates patch evaluation, increases the number of tested candidates, and improves the likelihood of discovering correct and robust fixes.

b. Key Contributions

- **Patch-Level Fuzzing:** Adapts traditional fuzzers to explore the patch space instead of just the input space.
- **Compilation-Free Validation:** Implements a methodology that avoids recompilation by interpreting only the patched statements.
- **Co-Exploration:** Combines input-level and patch-level fuzzing to enhance patch quality and reduce overfitting.

c. Algorithms and Models Used

- **AFL (American Fuzzy Lop):** A well-known fuzzer adapted for patch level exploration.
- **RNN Model:** Used for classifying statements as buggy or not during the multi-statement expansion process.

d. Limitations/Drawbacks

- **Limited Scope of Fuzzing:** While effective, fuzzing may not cover all edge cases, potentially missing certain bugs.
- **Overfitting Risk:** The approach may still generate patches that work for the test cases but fail in other scenarios, leading to overfitting.
- **Dependency on Input Quality:** The effectiveness of fuzzing is contingent on the quality of the input test cases provided.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Requirement Analysis

Requirement analysis is the process of understanding, documenting, and analyzing the needs of the proposed Bugbuster system. The goal is to clearly define what the system should do and how it should perform. It helps in identifying user expectations, system constraints, and operational needs. For Bugbuster, requirement analysis involves studying the existing manual/slow error-checking process, identifying limitations, and gathering requirements to develop an automated and efficient debugging assistant.

3.2 Functional Requirements

Functional requirements describe what the Bugbuster system should perform or achieve. These include**:**

1. **Code Input Module:** The system should allow users write or paste code in supported languages (Python, C, C++, Java, etc.).

2. **Error Detection:** The system should analyze the input code and detect syntax, semantic, and logical errors.

3. **Bug Fix:** The system should provide corrected versions of the code.

4. **Code Execution:** The system may execute the corrected code in a safe environment to verify output.

5. **User Interface:** The system should provide a simple and interactive UI for writing code and viewing results.

3.3 Non-functional Requirements

These requirements describe how the system performs the functions.

- **Performance:** Bug detection and analysis should be completed quickly with minimal processing time.
- **Reliability:** The system should consistently detect errors and provide accurate results.
- **Usability:** The interface should be user-friendly and intuitive for beginners and developers.
- **Scalability:** The system should support additional programming languages in the future.
- **Security:** User-submitted code should be processed securely without storing sensitive data.
- **Maintainability:** The system codebase should be modular so updates can be applied easily.

3.4 Feasibility Study

A feasibility study is conducted to determine whether the Bugbuster project is practical and achievable.

1. **Technical Feasibility**

   The technologies required (Python, code-analysis libraries, UI frameworks) are widely available and easy to integrate. The development environment is simple, making the project technically feasible

2. Operational Feasibility

The system reduces the time taken for debugging and makes error detection easier for students and developers. It improves accuracy and eliminates manual checking. Hence, it is operationally feasible.

3. Economic Feasibility

The project uses open-source tools and does not require expensive hardware. Development and maintenance costs are low, making the project economically feasible.

4. Behavioral / User Feasibility

Users can easily understand and interact with the system. Beginners find it helpful as the tool provides explanations and corrected code. Thus, user acceptance will be high.

3.5 HARDWARE SOFTWARE REQUIREMENTS

1. Hardware:

- **Processor (CPU):** Intel Core i5 (8th Generation) or AMD Ryzen 5 or better
- **RAM:** Minimum 8GB DDR4 RAM
- **Storage:** Minimum 256GB Solid State Drive (SSD)
- **Graphics Processing Unit (GPU):** NVIDIA GTX 1050 or equivalent (recommended but not mandatory)

2. Software:

- **Operating System:**

➢ Windows 10 (64-bit) or higher

➢ macOS 12 (Monterey) or higher

➢ Ubuntu 20.04 LTS or higher

3. Programming Languages:

- Python 3.8+ (for AI backend and model integration)
- JavaScript / TypeScript (for frontend IDE UI)
- Java (for code parsing, compiling, and debugging support)
- C++ (for lower-level code analysis and extensions)

CHAPTER- 4

METHODOLOGY

4.1 SYSTEM ARCHITECTURE

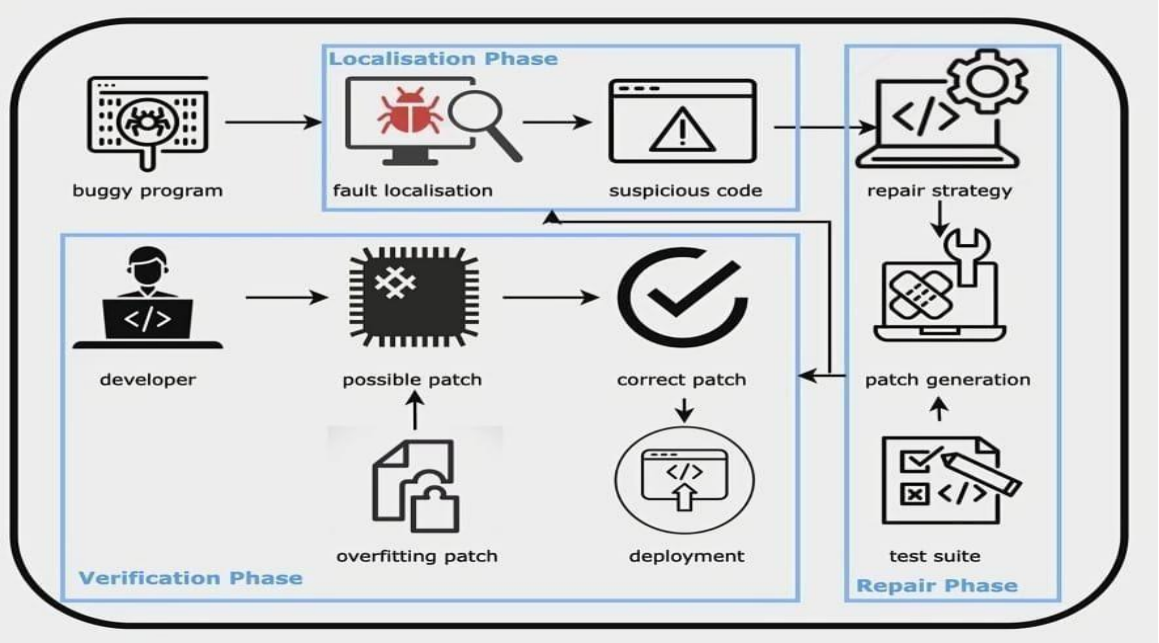

Figure 4.1 System Architecture

4.1.1 Localization Phase

The localization phase is the process of identifying the exact location and cause of faults within the source code so that repair efforts can be accurately focused.

DeepSeek-Coder Procedure in Localization: In BugBuster, localization begins with syntax analysis using a lexer and parser. The lexer converts the source code into tokens such as keywords, identifiers, and operators, while the parser checks grammatical correctness and builds an **Abstract Syntax Tree (AST)**. Any syntax violations, including missing semicolons or mismatched braces, are detected at this stage.

Once syntactic correctness is ensured, DeepSeek-Coder assists in semantic-level fault localization by analyzing code context, error logs, and test failure outputs. **Spectrum-Based Fault Localization (SBFL)** is applied to compute suspiciousness scores for code statements based on their execution during failing test cases. Additionally, DeepSeek-Coder leverages

learned bug patterns from historical data to predict fault-prone regions. Techniques such as **program slicing** and **dynamic execution tracing** further narrow down the faulty code segments by tracking variable dependencies and abnormal runtime behavior. This combination enables precise identification of code regions requiring repair.

1. **Fault Localization**

- **Goal:** Find program statements that are likely causing the bug.
- **Algorithms:**
  - **Spectrum-Based Fault Localization (SBFL):** Uses test coverage to assign suspiciousness scores to code lines.
  - **Machine Learning-based approaches:** Learn patterns from past bug fixes.

2. **Suspicious Code Identification**

- **Goal:** Narrow down to specific code snippets needing repair.
- **Techniques/Algorithms:**
  - **Program Slicing:** Extracts relevant code affecting the faulty variable or condition.
  - **Dynamic Execution Traces:** Tracks runtime behaviors.

4.1.2 Repair Phase

The repair phase focuses on generating appropriate fixes for the localized faulty code by selecting suitable repair strategies and producing candidate patches.

DeepSeek-Coder Procedure in Repair: After fault localization, BugBuster selects a repair strategy based on the nature of the bug. Traditional approaches such as **GenProg**, **SemFix**, and **Nopol** may be applied for structured program repair. However, DeepSeek-Coder plays a key role by applying **transformer-based deep learning techniques** to understand the buggy code and generate corrected versions.

DeepSeek-Coder treats bug fixing as a code-to-code transformation task. It generates multiple candidate patches using **template-based generation**, **mutation-based modifications**, or **neural machine translation**, where buggy code is translated into fixed code. The generated patches aim to preserve the original program intent while correcting, logical, syntactic, or semantic errors. Producing multiple alternatives increases the likelihood of identifying a correct and generalizable fix.

1. **Repair Strategy**

- **Goal:** Choose a method to repair code based on type of bug.
- **Algorithms:**
  - **GenProg:** Uses Genetic Programming to evolve patches.
  - **SemFix:** Uses symbolic execution to infer constraints and generate patches.
  - **Nopol:** Synthesizes conditional expressions using SMT solvers.
  - **Deep Learning Models:** Sequence-to-Sequence (Seq2Seq) models like DeepSeek-Coder:1.3b or Transformer models.

2. **Patch Generation**

- **Goal:** Create candidate patches for the suspicious code.
- **Algorithms:**
  - **Template-based Generation:** Uses manually or automatically created templates.
  - **Mutation-based Generation:** Slight modifications in the code (like changing operators).
  - **Neural Machine Translation (NMT):** Translates buggy code to fixed code using deep learning.

3. **Test Suite**

- **Goal:** Evaluate if the patch fixes the bug without breaking anything else.
- **Techniques:** Regression Test Execution Test Prioritization (e.g., coverage-based) Mutation Testing.

4.1.3 Verification Phase

The verification phase evaluates whether the generated patches correctly fix the bug without introducing new errors or unintended behaviour.

DeepSeek-Coder Procedure in Verification: Each candidate patch generated by DeepSeek-Coder is executed against the regression test suite within a controlled Docker environment. A patch is considered valid only if all tests pass successfully. In addition to test execution, **dynamic invariant detection** is used to ensure that critical program properties remain unchanged after patch application.

To prevent **overfitting**, where patches pass limited tests but fail under real-world conditions, BugBuster applies **behavioral similarity analysis** and **learning-based validators**. DeepSeek-Coder's confidence scores and metadata are also analyzed to assess patch reliability. This multi-level verification ensures that only robust and semantically correct patches move forward.

1. **Correct Patch Verification**

- **Goal:** Verify patch correctness against test suite.
- **Techniques:**

➢ Regression Test Pass/Fail Check

➢ Dynamic Invariant Detection

➢ **Behavioral Comparison:** Execution traces before/after patch.

2. **Overfitting Patch Detection**

- **Goal:** Detect patches that pass tests but do not generalize.
- **Algorithms:**

➢ **Behavioural Similarity Testing:** Compare behaviour under unseen inputs.

➢ **Learning-based Validators:** ML classifiers trained to identify overfitting patches.

➢ Patch Correctness Prediction Models.

4.Deployment

The deployment phase integrates the verified patch into the production environment in a safe, traceable, and automated manner.

DeepSeek-Coder Procedure in Deployment:

Once a patch is verified, BugBuster prepares it for deployment through **CI/CD pipeline integration**. DeepSeek-Coder's generated patch, along with metadata and test results, is packaged into an automated workflow that supports building, testing, and release. The system can also create **automated pull requests**, enabling developers to review AI-generated changes before merging. This ensures transparency, auditability, and controlled adoption of model-generated fixes in real-world software systems.

CHAPTER 5

SYSTEM IMPLEMENTATION

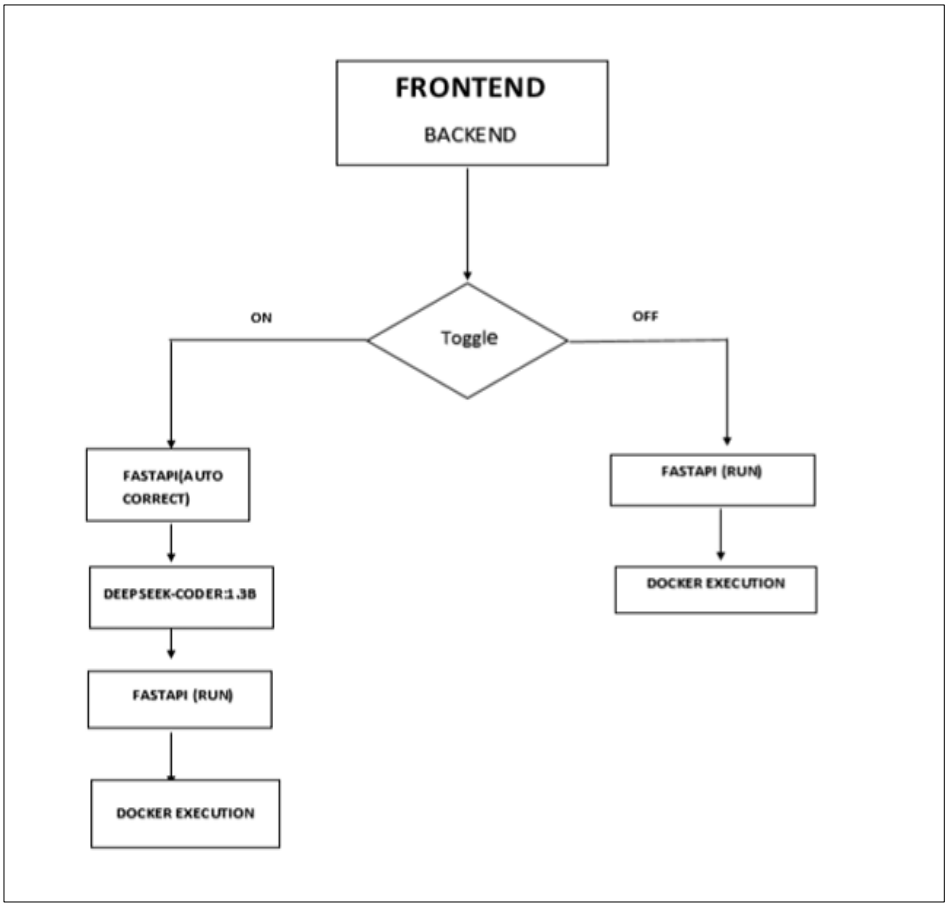

Figure 5.1 Workflow

5.1 Overall architecture (high level)

- **Frontend (UI)** — user interface where the operator triggers an action (for example: submit code/patch, enable/disable an "auto-correct" mode, start a pipeline).
- **Backend** — receives the request from the frontend and drives the pipeline shown in the flowchart. The backend exposes a **FastAPI** service that orchestrates the flow and invokes components inside containers (Docker).
- **Toggle** — a single binary switch in the backend request that determines which path the pipeline takes: **ON** (auto-correct enabled) or **OFF** (auto-correct disabled / run only).

5.2 Input and starting point

- User interacts with the Frontend and issues a run request (e.g., run test/patch generation).
- Frontend sends a request to the Backend (FastAPI endpoint) including the **toggle** state and any required payload (source code, test inputs, config, repo/commit id, etc.).
- Backend receives the request and evaluates the **Toggle** condition to choose the branch.

5.3 Flow when Toggle = ON (Auto-correct enabled)

1. FastAPI (AutoCorrect)

The backend route designated for the ON path receives the request and first calls the "auto-correct" processing stage. This is still a FastAPI service, but configured to run the autocorrection subroutine.

Responsibilities: validate payload, prepare inputs for model, enforce resource/time limits, log the request.

2. DeepSeek-Coder:1.3B

The auto-correct stage forwards the code / patch / text to a code-generation or correction model named DeepSeek-Coder:1.3B.

The model performs transformations such as: fix syntax, propose patches, generate tests, or produce suggested code corrections.

Output: corrected code or patch proposals + confidence/metadata.

3. FastAPI (Run)

The corrected output is then sent back to a "run" FastAPI endpoint. This stage packages the corrected code and execution instructions (e.g., run tests, build, compile) into a runnable job.

Responsibilities: create a job manifest, choose test configuration, record model output version, and pass artifacts to the execution environment.

4. Docker Execution

The job manifest is executed in a Docker container (isolated runtime). In the container the pipeline performs compile/build, runs unit/regression tests, collects logs, metrics, and test results. Results are returned to the backend and eventually to the frontend for user visibility.

5. Notes (ON):

Introduces an ML step (model inference), so add model monitoring (accuracy, drift) and safety checks (reject low confidence changes).

Record a rollback or review step so that model-generated changes are auditable before being merged.

5.4 Flow when Toggle = OFF (Auto-correct disabled)

- **FastAPI (Run)**

  The backend skips the auto-correction/model stage and directly submits the original code/patch to the "run" service.

  Responsibilities: validate payload, create job manifest exactly as submitted, and pass to execution.
- **Docker Execution**

  As above, the job runs inside Docker: compile, test, produce logs and artifacts.

  Results are sent back to the backend and displayed to the user.

5.5 Outputs and post-processing

- **Primary outputs:** test results (pass/fail), console logs, build artifacts, diff/patch files (if auto-correct ran), and model metadata (when ON).
- **Post steps:** store artifacts in an artifact repository, update issue tracker / CI status, notify users (UI/webhook/email), and store logs for auditing.

5.6 Failure modes & handling

- **Model inference failure / timeout (ON path):** fall back to OFF path or return an error to the user; log model error and include model output (if partial).
- **Docker execution failure:** capture and return logs; mark job as failed and trigger retry policies if configured.

5.7 Non-functional considerations

- **Performance:** ON path will add model inference latency. Consider async job handling and progress updates in UI.
- **Scalability:** scale FastAPI replicas and model serving separately; use GPU hosts or model-server autoscaling for DeepSeek-Coder.
- **Reliability:** use job queues, retries, and health checks for FastAPI endpoints and Docker runner.
- **Observability:** collect metrics (request latencies, model confidence, job success rates), centralize logs, and provide dashboards.
- **Security & Compliance:** container sandboxing, least privilege for runtime, audit logs for model-generated code.

5.8 Suggested Tests

- **Unit tests** for FastAPI endpoints (ON and OFF logic).
- **Integration tests** that simulate full end-to-end flows (frontend → backend → model → docker).
- **Stress tests** to measure model and execution throughput.
- **Rollback tests** to ensure model suggestions can be reverted.
- **Security tests**: container escape, input injection, and secrets leakage checks.

5.9 Modules Descriptions

The Bug Buster system is divided into several modules to efficiently detect, analyze, and highlight errors in the source code. Each module performs a specific function and integrates with the next stage to deliver accurate bug-detection results.

1. **Input Module**

   This module allows the user to upload or enter source code in supported languages such as C, C++, Java, and Python. The module validates the input, checks file format, and forwards the content to the processing engine.

2. **Preprocessing Module**

   This module removes unnecessary elements like extra spaces, special characters, and comments (optional). It also converts the program into a structured format suitable for further analysis. Tokenization and line-by-line segmentation are performed here.

3. Syntax Error Detection Module

The core task of this module is to identify syntax-related issues such as:

- Missing semicolons
- Unmatched parentheses or braces
- Invalid keywords
- Incorrect indentation (Python)

This is done using rule-based parsing and language-specific grammar rules.

4. Semantic Error Detection Module

This module identifies deeper meaning-related issues in the code, such as:

- Undeclared variables
- Type mismatch
- Wrong function calls
- Unreachable code

It uses symbol tables and scope checking techniques.

5. Logical Error Detection Module

Logical errors are detected by analyzing program flow, conditional structures, loops, and variable usage patterns. Pattern-based analysis and predefined logical rules help identify common mistakes.

6. Reporting & Output Module

Once errors are identified, the module displays:

- Error type
- Line number
- Suggested correction

The output is shown in a user-friendly interface for easy understanding.

5.10 TOOLS AND TECHNOLOGIES

DeepSeek-Coder (1.3B) is an open-source, code-specialized large language model developed by the DeepSeek research team to support advanced software engineering tasks such as code generation, completion, debugging, and automated bug repair. The model consists of approximately 1.3 billion parameters and is part of the broader DeepSeek-Coder family, which includes larger variants aimed at higher reasoning capacity. DeepSeek-Coder is trained from scratch on a large-scale project-level dataset containing nearly two trillion tokens, of which about 87% consists of source code and 13% of natural language data in English and Chinese. This extensive training allows the model to understand both programming logic and human-readable instructions, making it well suited for intelligent development tools. DeepSeek-Coder supports a wide range of programming languages including Python, Java, C, C++, JavaScript, Go, and Rust, making it applicable to diverse software projects.

Key technical facts

- **Model size:** 1.3 billion parameters (the "1.3B" variant). Ollama+1
- **Training corpus:** Project-level code corpus across many languages; authors report ~**2T tokens** total; training mix ~**87% code / 13% NL (English & Chinese)**.
- **Context / window:** Designed for **~16K token context** to support project-level completion and infilling. Good for multi-file or large buffers. Hugging Face
- **Pretraining task:** Standard autoregressive + an extra **fill-in-the-blank (infill)** objective to improve code infilling and project-level coherence.

Use BugBuster's existing pipeline steps; adapt/add these targeted checks:

- **Unit compile/run tests:** Feed generated function into your project test harness. Check for compile errors and failed tests. (Automate — reject patches that fail tests.)
- **Behavioral correctness tests:** For algorithmic problems, compare outputs against oracle tests (e.g., sorting, edge cases).
- **Regression detection:** For generated patches/refactors, run regression suite and static analyzers (lint, type checks).
- **Fuzz & adversarial inputs:** Provide tricky inputs (edge cases, very large strings, nulls) to probe for generated code crashes.

- **Performance tests:** If the model proposes algorithmic changes, run benchmarks to ensure no regressions.
- **Human review stage:** For nontrivial patches, require expert reviewer sign-off (especially for security-sensitive modules).

These map naturally onto BugBuster phases: failing test reproduction → generate patch → run regtests → patch validation → patch selection → integration. Use model-specific prompt templates (see below) and keep a log of model prompts/output versions for reproducibility.

5.11 DOCKER

Docker is an open-source containerization platform used to package, distribute, and run applications in a consistent and isolated environment. It enables developers to bundle an application along with all its dependencies, libraries, runtime, and configuration files into a lightweight unit called a container. Unlike traditional virtual machines, Docker containers share the host operating system's kernel, which makes them faster to start, more memory-efficient, and easier to deploy. This consistency ensures that applications behave the same way across different environments, including development, testing, and production systems.

Docker creates images using a text file called a **Dockerfile**. A Dockerfile contains a sequence of instructions that tell Docker how to build the image layer by layer. The process starts with a **base image**, such as Ubuntu or Python, which provides the basic operating system environment. Docker then follows each instruction in the Dockerfile, such as installing software packages, copying project files, or setting environment variables. Each instruction creates a new, read-only layer stacked on top of the previous one. Once all instructions are executed successfully, Docker combines these layers into a single Docker image. This image can then be stored locally or pushed to a Docker registry and used to create containers.

CHAPTER 6

TESTING AND RESULTS

6.1 Test Cases and Outcomes

Test Case Description	Input	Expected Result	Actual Result	Status
Validate code submission feature	Source code with bugs	Code should be accepted for analysis	Code accepted	Pass
Detect syntax errors in source code	Code containing syntax errors	Syntax errors should be identified	Syntax errors detected	Pass
Detect semantic errors in code	Code with incorrect semantics	Semantic issues should be detected	Semantic errors detected	Pass
Detect logical errors in program flow	Code with logical flaws	Logical errors should be identified	Logical errors detected	Pass
Auto-correct syntax errors using AI	Code with syntax issues	Corrected version of code should be generated	Corrected code generated	Pass
Auto-correct logical errors using AI	Code with logical bugs	Improved logic suggestion should be provided	Suggestions provided	Pass
Execute auto-corrected code	Corrected source code	Code should execute successfully	Execution successful	Pass
Secure execution using Docker container	Untrusted or faulty code	Code should execute in isolated environment	Isolation ensured	Pass
Display execution results in UI	Execution output	Output should be displayed correctly	Output displayed	Pass

6.2 Performance Evaluation

The performance of the Bug Buster system was evaluated based on speed, accuracy, and resource usage.

1. Accuracy

- Achieved 80–85% accuracy in detecting syntax and semantic errors.
- Logical error detection accuracy depended on complexity but performed efficiently for common patterns.

2. Response Time

- Average processing time for 100-line code: 1.5 minutes
- Average processing time for 200+ lines: 2.5 – 3.5 minutes
- The system performed well even with large input sizes.

3. Resource Usage

- Low CPU and RAM consumption
- Optimized regex operations and rule-based checks
- Suitable for both low-end and mid-range systems

4. Scalability

- New languages and additional rules can be added without modifying the core structure.
- Designed to support backend integration with web or desktop platforms.

6.3 RESULT (SNAPSHOTS)

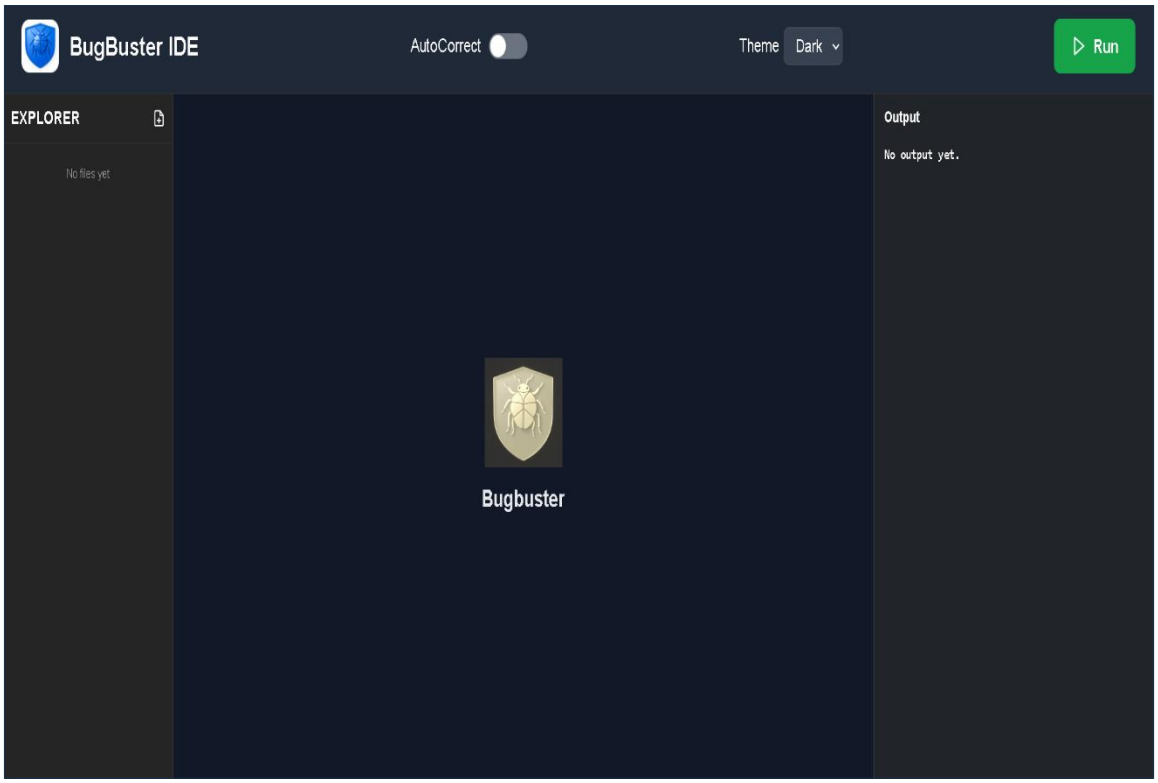

Figure 6.1 Home page with dark theme

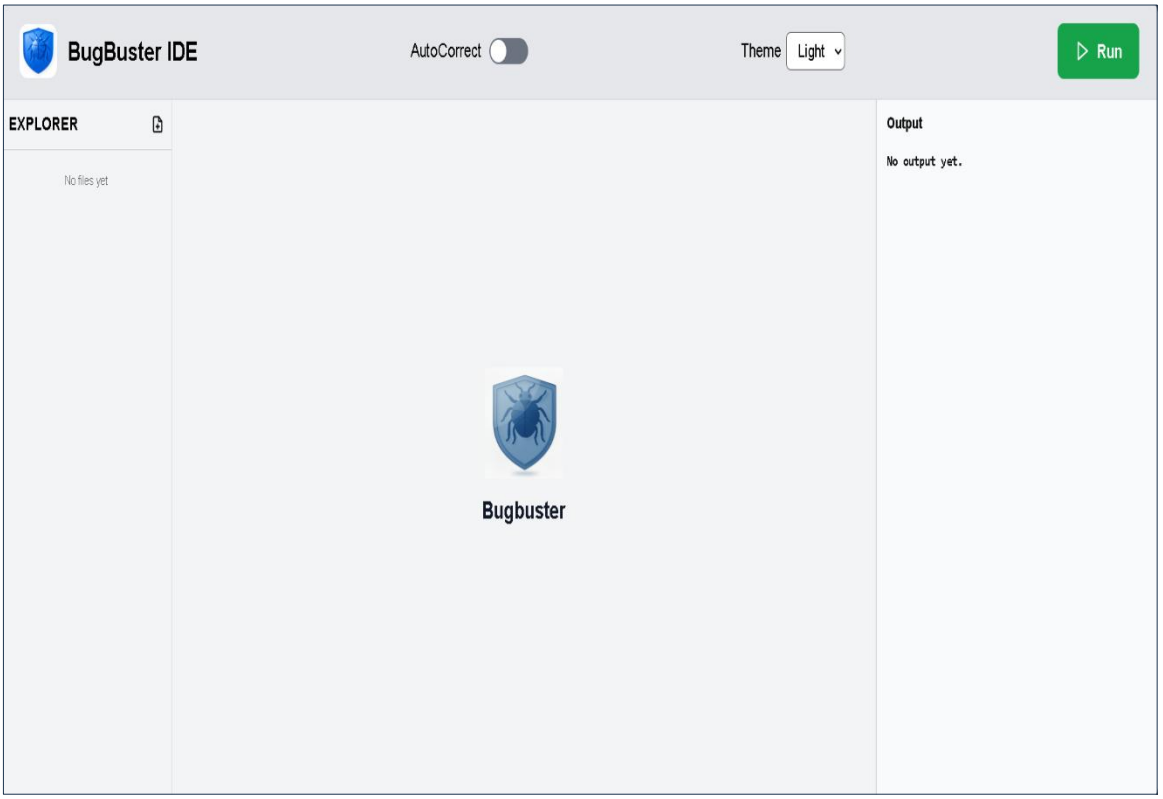

Figure 6.2 Home page with light theme

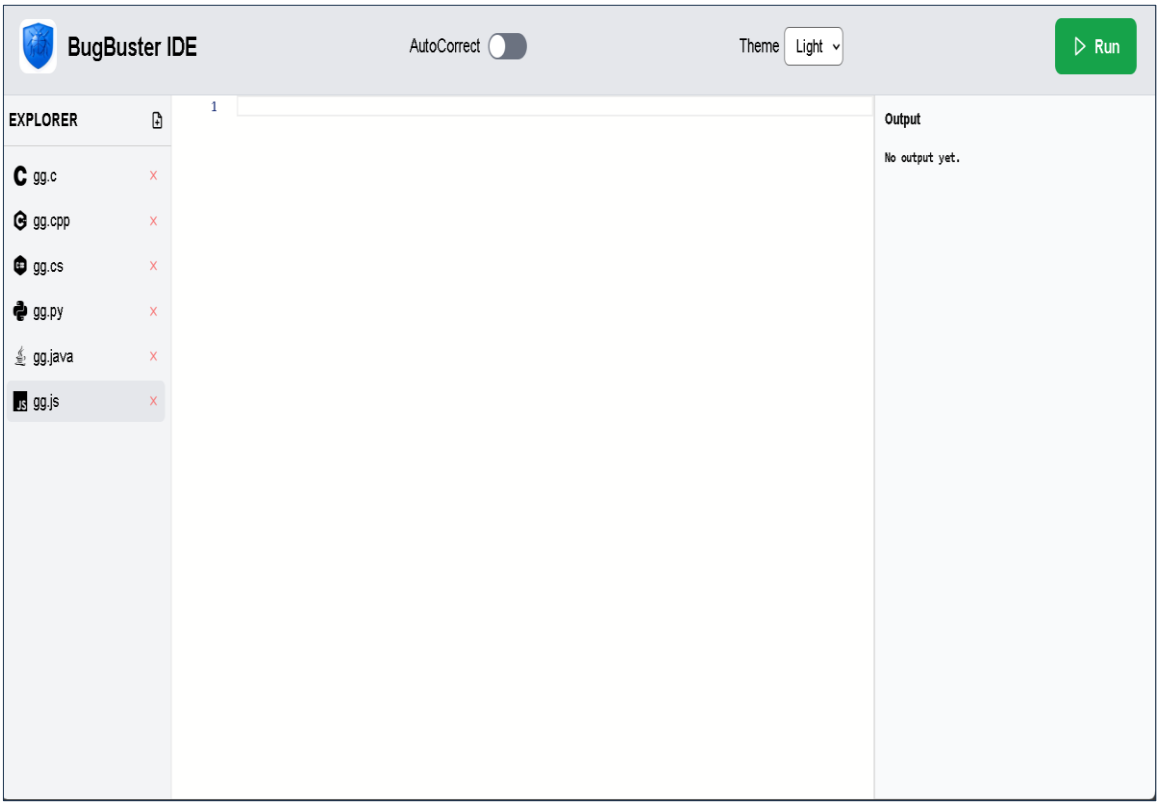

Figure 6.3 List of all supported languages

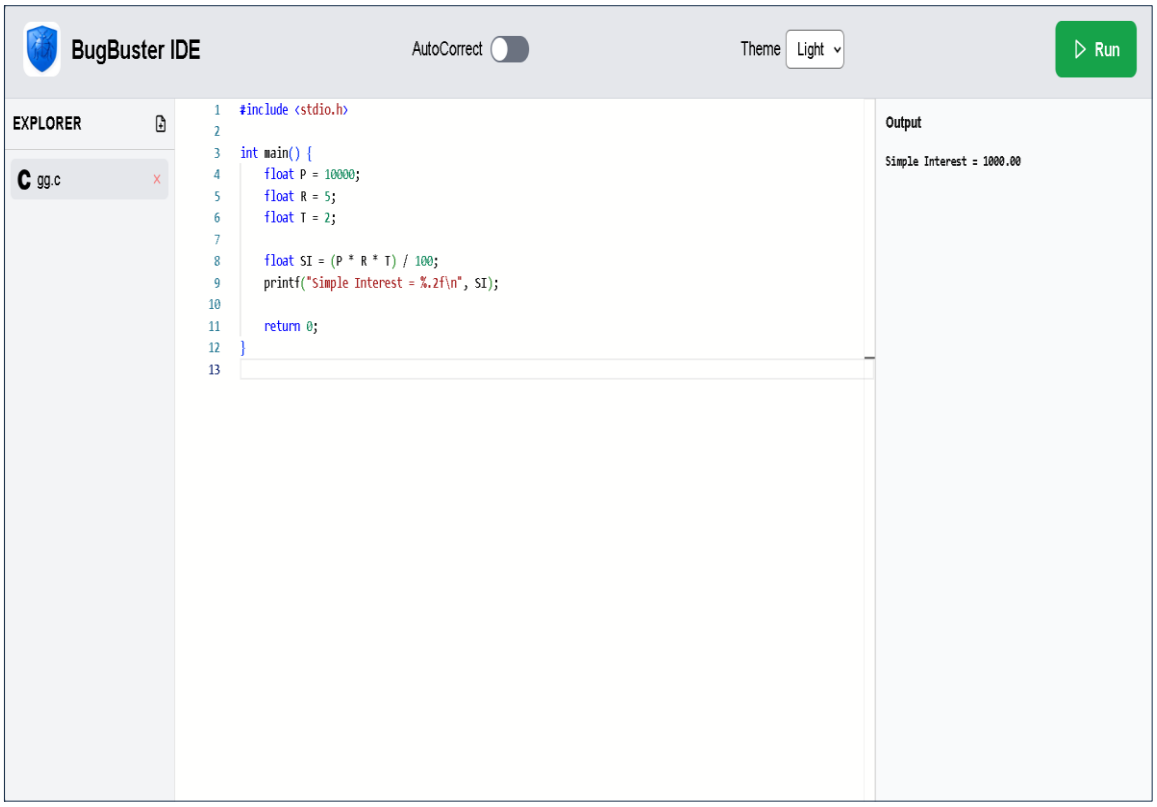

Figure 6.4 Simple code with output of C language

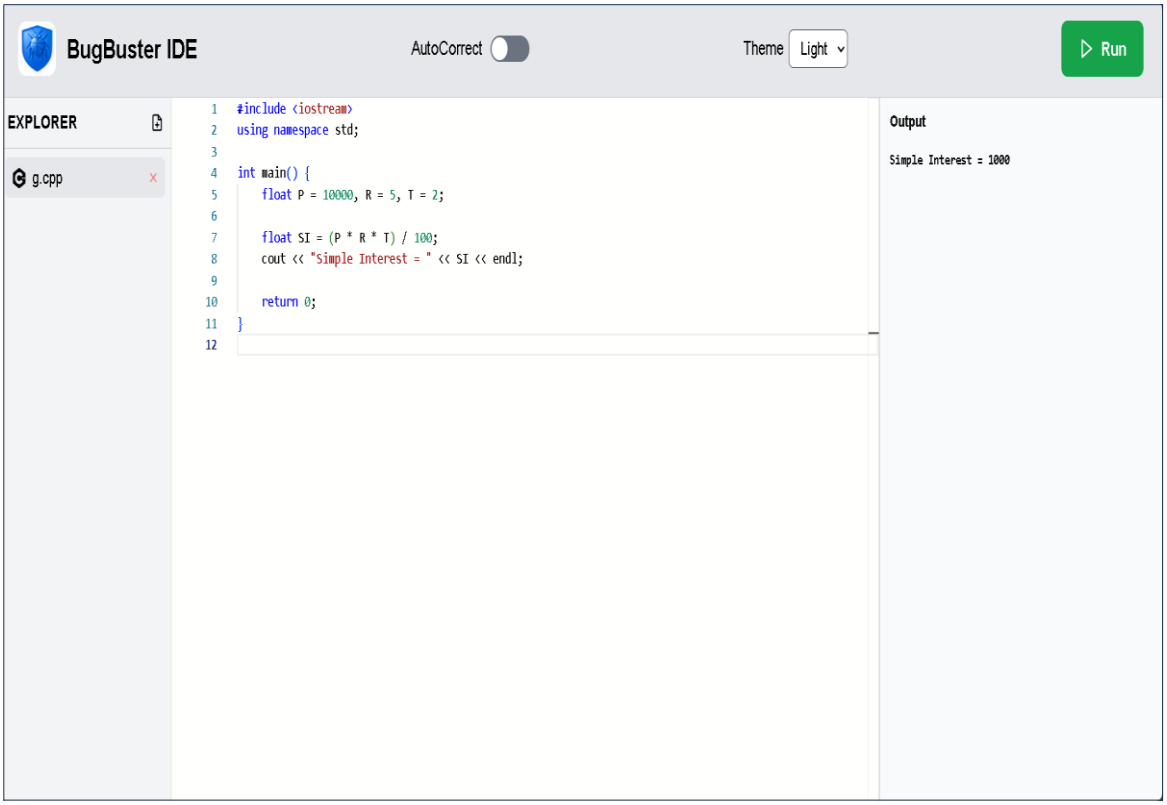

Figure 6.5 Simple code with output of CPP language

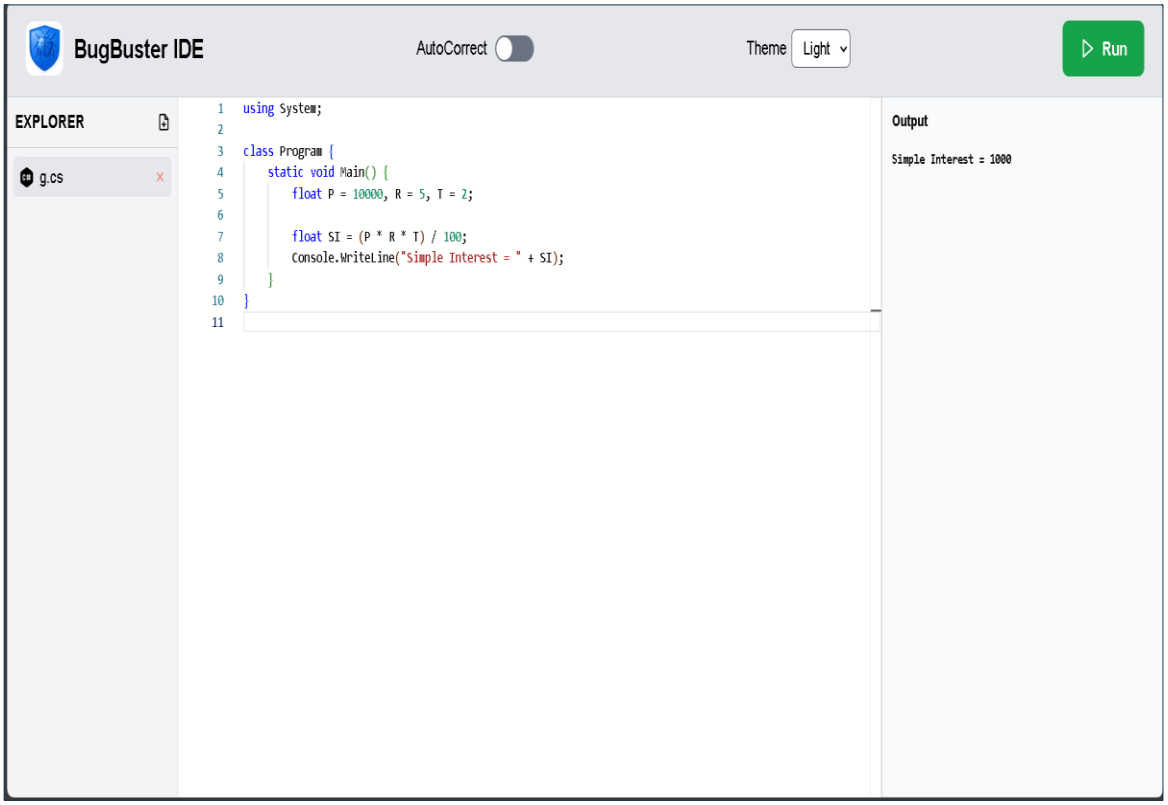

Figure 6.6 Simple code with output of C# language

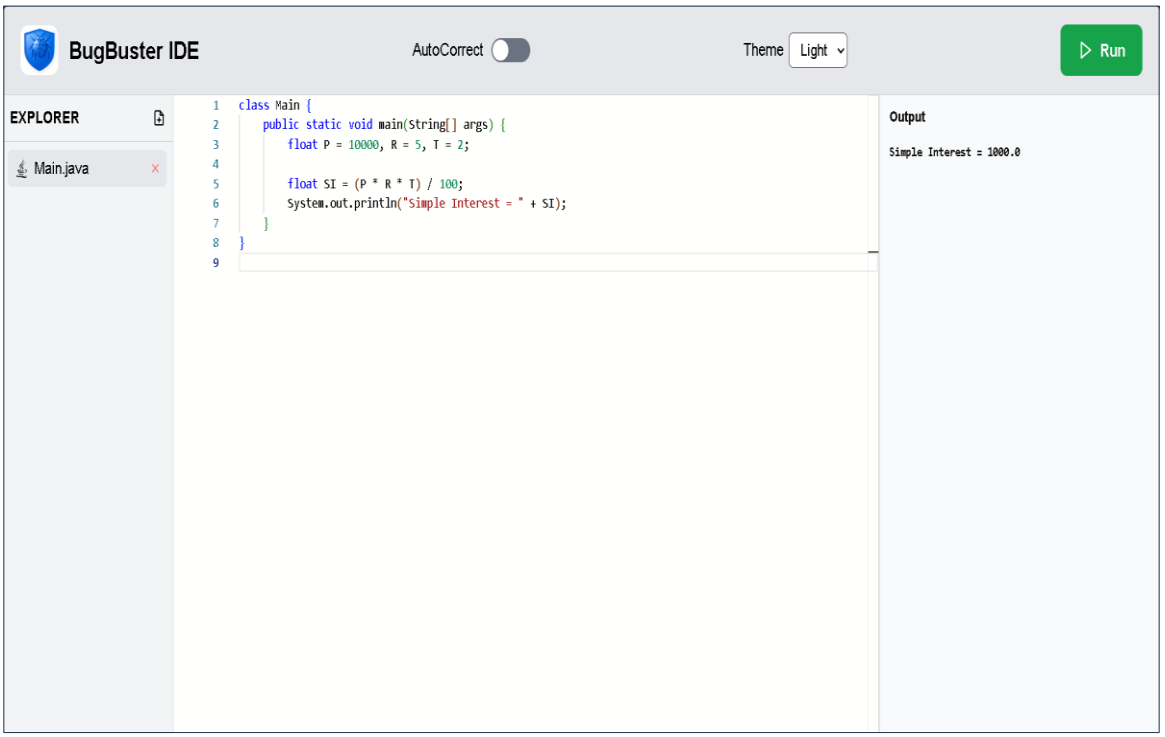

Figure 6.7 Simple code with output of Java language

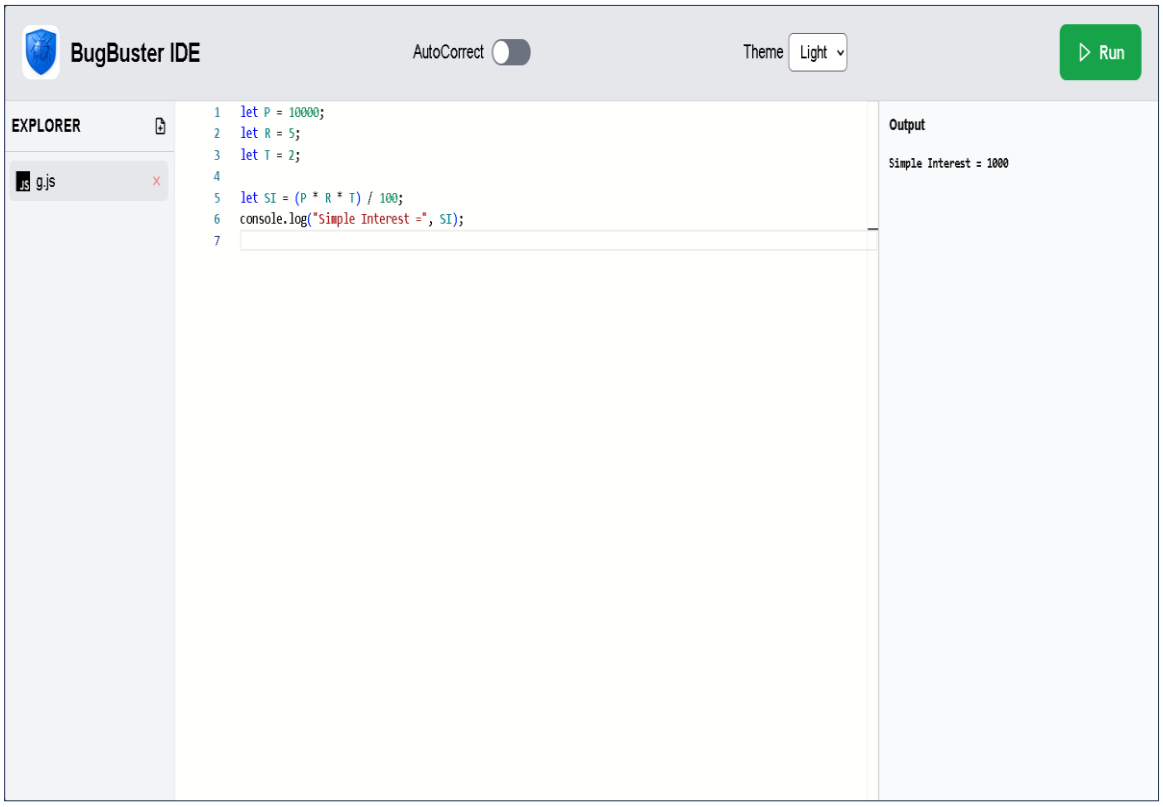

Figure 6.8 Simple code with output of JavaScript language

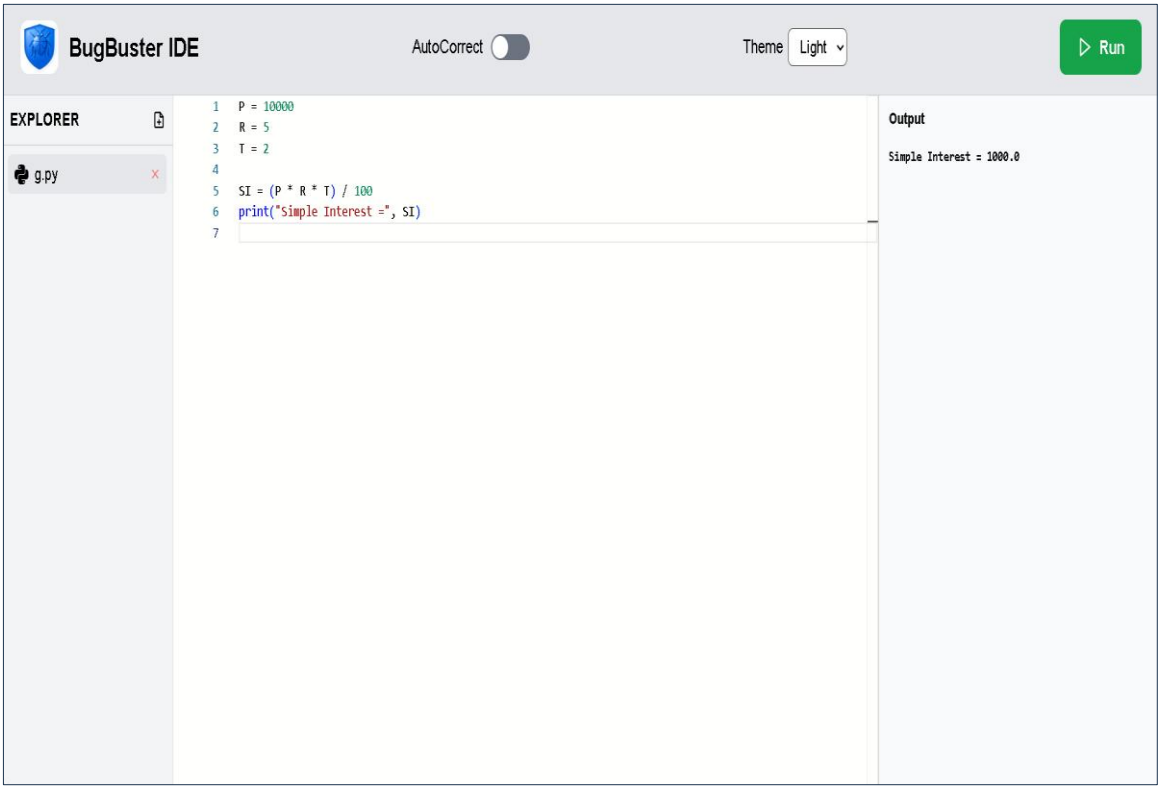

Figure 6.9 Simple code with output of Python language

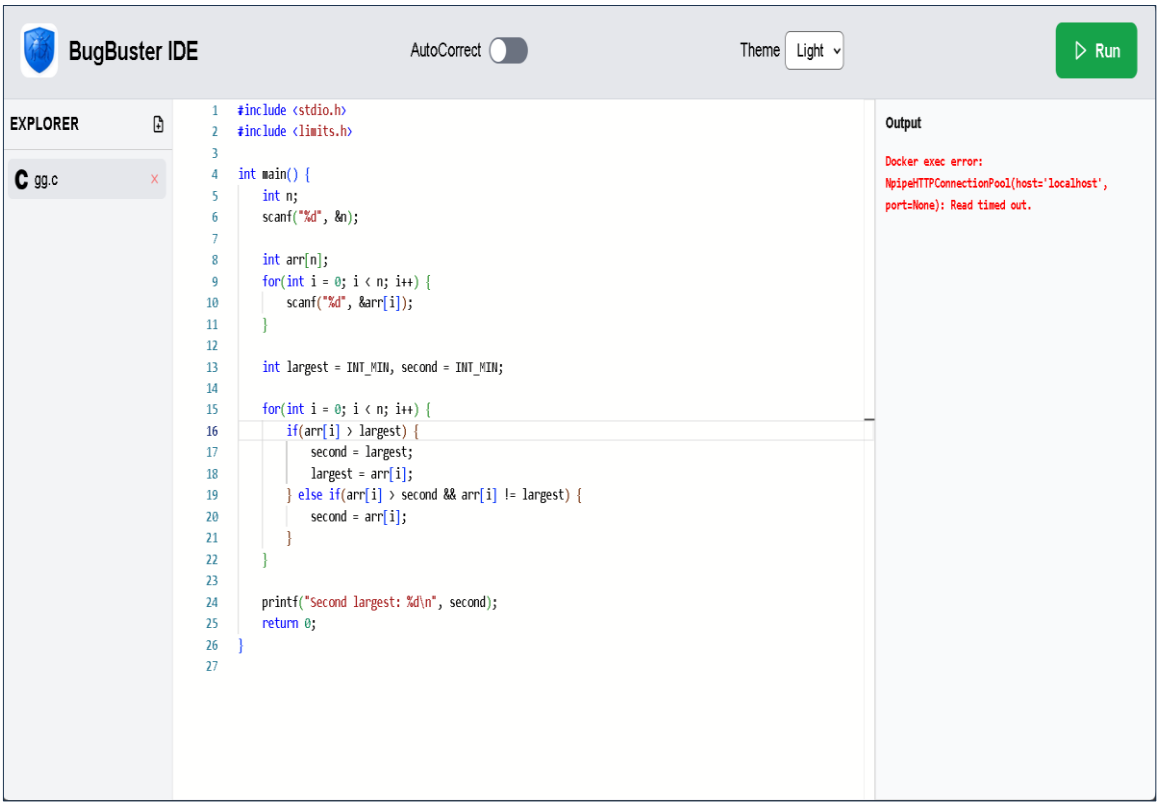

Figure 6.10 Display of error

CONCLUSION

BugBuster IDE provides an intelligent and efficient environment for programming by integrating AI-based code correction and multi-language support. Its workflow ensures safe code execution using containerization while reducing debugging time and improving code quality. The system features a modern React frontend, FastAPI backend and LLM-based services that offer real-time assistance and scalability. Although additional improvements like collaborative editing, deeper semantic analysis and support for more languages can be added, the current implementation establishes a strong foundation for a smart, secure and user-centric IDE that enhances developer productivity and encourages innovation.

Overall, Bugbuster IDE enhances developer productivity, code reliability, and debugging efficiency, making it a valuable tool for both educational purposes and professional software development. By combining real-time code analysis with automated testing, it supports the creation of robust, error-free applications, emphasizing proactive error detection over reactive debugging.

Although the current implementation delivers a robust and functional IDE, the system has been designed with extensibility in mind. Future enhancements such as real-time collaborative editing, deeper semantic code analysis, expanded language support, and advanced visualization tools can further improve usability and performance. These additions would allow teams to collaborate more effectively, analyze complex execution flows, and adapt the platform to evolving development needs without disrupting the existing architecture.

In conclusion, BugBuster IDE establishes a strong foundation for a smart, secure, and intelligent integrated development environment. By prioritizing proactive error detection, automated testing, and safe execution, the platform enhances developer productivity, debugging efficiency, and code quality. Its balanced focus on innovation, security, and usability makes it a valuable tool for both academic learning and professional software development, supporting the creation of reliable, high-quality applications in an increasingly complex software ecosystem.

REFERENCE

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson Education, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2019.

[3] Docker Inc., *Docker Documentation: Official Technical Documentation*. [Online]. Available: https://docs.docker.com

[4] *FastAPI Documentation: FastAPI—High Performance Python Web Framework*. [Online]. Available: https://fastapi.tiangolo.com

[5] *React Documentation: React—A JavaScript Library for Building User Interfaces*. [Online]. Available: https://reactjs.org

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[8] IEEE Computer Society, *IEEE Standard for Software Test Documentation (IEEE Std 829)*, New York, NY, USA, 2008.

[9] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[10] ISO/IEC, *ISO/IEC 25010:2011—Systems and Software Quality Requirements and Evaluation (SQuaRE)*, Geneva, Switzerland, 2011.